

Capstone Project 3

Serverless Task Management System

GitHub Repository: <https://github.com/Amasha456/capstone-project-3-serverless>

Submitted by: Nawanjana Hasaranga Fonseka Neththikumara Haththellage

Architecture Diagram

```
Client
|
v
API Gateway (API key required on write access)
|
+-- POST /tasks --> createTask Lambda --> DynamoDB (Tasks table)
|                                     \-- SQS (task-notifications queue)
|
+-- GET /tasks --> listTasks Lambda --> DynamoDB (Tasks table)

SQS task-notifications queue
|
+-- triggers --> processNotification Lambda
                  (on repeated failure, after 3 attempts, message moves to
                   the task-notifications-dlq Dead Letter Queue instead of
                   being silently dropped)

EventBridge (daily schedule, rate(1 day))
|
+-- triggers --> exportReport Lambda --> DynamoDB (read all tasks)
                                      --> S3 (writes a CSV report)

Four Lambda functions, three distinct trigger types (HTTP via API Gateway,
event-driven via SQS, and scheduled via EventBridge).
```

Capstone Project 3: Serverless Task Management System

Overview

An event-driven serverless REST API for managing tasks, built entirely on AWS Lambda, API Gateway, DynamoDB, SQS, S3, and EventBridge, deployed with AWS SAM and automatically built/tested/deployed via GitHub Actions.

Business Problem

A lightweight backend for a task-tracking tool that needs to scale to zero when idle (no cost, no servers to manage) and scale automatically under load, while also handling background work (sending notifications, producing periodic reports) without blocking the API response to the user.

Technologies Used

- **Compute:** AWS Lambda (Node.js 20.x)
- **API:** Amazon API Gateway (REST API, API key + usage plan for authentication/rate limiting)
- **Database:** Amazon DynamoDB (pay-per-request)
- **Messaging:** Amazon SQS (main queue + Dead Letter Queue)
- **Storage:** Amazon S3 (generated CSV reports)
- **Scheduling:** Amazon EventBridge (daily rule)
- **IaC / deployment:** AWS SAM (Serverless Application Model)
- **CI/CD:** GitHub Actions
- **Testing:** Node.js's built-in test runner (`node --test`), no external test framework dependency

Prerequisites

- AWS account with billing enabled
- AWS CLI v2, configured (`aws configure`)
- AWS SAM CLI
- Node.js 20.x (for running tests locally; not required for deployment,

since the Lambda runtime already bundles the AWS SDK v3)

Installation Instructions

```
git clone <this-repo-url>
cd capstone-project-3-serverless
sam build
```

Deployment Instructions

```
sam deploy --guided
```

On first run this asks for a stack name, region, and whether to save the

answers to `samconfig.toml` (recommended - subsequent deploys are then just

`sam deploy`). After deploying, the API URL and other resource identifiers

are printed as CloudFormation stack outputs:

```
aws cloudformation describe-stacks --stack-name capstone-serverless-tasks \
  --region eu-central-1 --query "Stacks[0].Outputs"
```

API Documentation

Base URL: `https://.execute-api.eu-central-1.amazonaws.com/prod`

Method	Path	Auth required	Description
POST	/tasks	Yes (x-api-key header)	Create a task. Body: <code>{"title": "..."} </code>
GET	/tasks	No	List all tasks

Example:

```
curl -X POST https://<api-id>.execute-api.eu-central-1.amazonaws.com/prod/tasks \
  -H "x-api-key: <your-api-key>" \
  -H "Content-Type: application/json" \
  -d '{"title": "Buy milk"}'
```

The API key is retrievable via:

```
aws apigateway get-api-keys --include-values --region eu-central-1
```

Testing Instructions

- **Unit tests** (validation logic and CSV-building logic, no AWS calls

needed): `node --test tests/unit.test.js` - 8 tests, all passing. These

also run automatically in the CI/CD pipeline before every deploy.

- **End-to-end, against the live deployment:**

- Create a task via `POST /tasks` and confirm it's returned with an id.

- List tasks via `GET /tasks` and confirm the new task appears.

- Confirm the notification function fired by checking its CloudWatch Logs

for a Notification: task "... " was created line.

- **Failure/DLQ path:** create a task with the word "fail" in its title

(a deliberate test hook built into processNotification) - this

simulates a downstream failure. Confirm in CloudWatch Logs that it

retried 3 times, then confirm the message landed in the DLQ:

```
aws sqs get-queue-attributes --queue-url --attribute-names ApproximateNumberOfMessages
```

- **Scheduled report:** manually invoke exportReport with

```
aws lambda invoke --function-name out.json and
```

confirm a CSV appears in the S3 bucket under reports/.

Security Considerations

- Write access (POST /tasks) requires an API key tied to a usage plan

with request throttling (rate limit 10 req/s, burst 20) - this both

authenticates and rate-limits the write path.

- Each Lambda function has its own least-privilege IAM role, generated

automatically by SAM's policy templates (e.g. DynamoDBCrudPolicy,

SQSSendMessagePolicy) rather than a single broad role shared across

functions.

- **Known limitation:** read access (GET /tasks) is intentionally left

public with no API key, since this demo has no concept of user accounts;

a real deployment would put both routes behind proper authentication

(e.g. Amazon Cognito) rather than a shared API key.

- **Known limitation:** as with Project 1, AWS CLI/CI deployment access

used root account credentials due to time constraints; a production setup

should use a scoped IAM user or an OIDC-based GitHub Actions role instead.

- Secrets (AWS credentials for CI/CD) are stored as GitHub Actions

repository secrets, never committed to the repo.

Monitoring Strategy

Every Lambda function automatically logs to its own CloudWatch Logs group

(no extra setup needed - this is built into the Lambda service). The processNotification function's logs were used directly during testing to confirm both the success path and the retry/failure path. CloudWatch metrics (invocation count, errors, duration, throttles) are available per function and per API route without additional configuration.

Cost Considerations

This architecture is close to zero-cost at low/demo traffic levels, since every component is either free-tier eligible or billed per-request rather than per-hour:

Component	Pricing model	Approx. cost at this project's usage
Lambda (4 functions)	Per request + per ms compute	Within free tier (1M requests/month free)
API Gateway	Per request	~\$3.50 per million requests (well within free tier at demo volume)
DynamoDB	Pay-per-request	Within free tier (25 free reads/writes)
SQS	Per request	Within free tier (1M requests/month free)
S3	Per GB stored + per request	Negligible for a few small CSV files
EventBridge	Per rule/event	Negligible (1 rule firing once daily)

Unlike Project 1's EKS cluster (which bills hourly for the control plane and nodes regardless of traffic), this serverless architecture has **no idle cost** - nothing was running (and therefore nothing was billing) between testing sessions, which is a genuine cost advantage of this architecture over the Kubernetes one for a low/variable-traffic workload.

Cleanup Instructions

```
sam delete --stack-name capstone-serverless-tasks --region eu-central-1
```

This removes every resource the stack created (Lambda functions, API

Gateway, DynamoDB table, SQS queues, EventBridge rule) except the S3 bucket if it still contains objects - empty it first if needed:

```
aws s3 rm s3://capstone-task-reports-448049824779 --recursive
```

Known Limitations

- No authentication on the read (GET /tasks) endpoint.
- Used AWS root account credentials rather than a scoped IAM user (see Security Considerations).
- No pagination on GET /tasks - a Scan against DynamoDB doesn't scale well past a few thousand items; a production version should use Query with a proper access pattern and pagination.
- The Lambda runtime (nodejs20.x) is approaching AWS's deprecation timeline; a future update should move to nodejs22.x.
- No staging environment - the CI/CD pipeline deploys straight to the only environment on every push to main.

Project 3 Report: Architecture, Security, Scalability, Reliability, Limitations, Cost

Architecture

The system is a fully event-driven serverless application with four Lambda functions, each with a single, clear responsibility, triggered three different ways: two behind API Gateway (HTTP), one behind SQS (event), and one behind EventBridge (schedule). This separation means each piece scales, fails, and can be redeployed independently - creating a task and generating a report are entirely decoupled processes that never block each other.

DynamoDB was chosen over a relational database for its pay-per-request pricing and zero idle cost, which matches a low/unpredictable-traffic workload far better than a provisioned database would.

Security

Write access is gated behind an API Gateway API key tied to a usage plan, which both authenticates requests and rate-limits them (10 req/s, 20-request burst) - a single mechanism doing double duty. Each function has its own narrowly-scoped IAM role rather than one shared role, generated automatically by SAM's policy templates (e.g. the notification function can only read from its one queue; it has no DynamoDB or S3 access at all, since it doesn't need any).

The most significant gap, documented honestly rather than hidden, is that deployment access (both the local AWS CLI and the GitHub Actions pipeline) used the AWS account's root credentials, the same tradeoff made in Project 1 under the same time constraint. The read endpoint is also intentionally left open with no auth, since this demo has no user-account concept - a real multi-tenant version would need per-user authentication (Cognito) on both routes, not just the write path.

Scalability

Every component in this architecture scales automatically and independently with no configuration: Lambda scales concurrency to match incoming request volume, DynamoDB's pay-per-request billing mode scales read/write capacity automatically, and SQS/API Gateway both handle bursts natively. This is a structurally different scalability story than Project 1's Kubernetes deployment, where a real, observed limit was hit (node pod-count capacity) during load testing. Nothing analogous was possible to hit here at the traffic levels tested - the closest thing to a scalability concern is the GET /tasks endpoint's use of a DynamoDB Scan, which reads every item in the table regardless of how many tasks exist; this works fine at demo scale but would need to become a Query against a proper access pattern (e.g. by status or date) well before the table reached meaningful size.

Reliability

The clearest reliability feature built and **tested** (not just configured) is the SQS Dead Letter Queue on the notification path. A deliberate test hook was built into processNotification so a task titled with the word "fail" throws an error on purpose; this was used to confirm, via CloudWatch Logs, that SQS retried the failed message three times (matching the queue's configured maxReceiveCount) roughly 30 seconds apart before routing it to the task-notifications-dlq queue, confirmed directly via `aws sqs get-queue-attributes`. This means a broken downstream notification service can't silently swallow events forever, and a message that does eventually fail is preserved for manual inspection or reprocessing rather than lost.

Limitations

- No authentication on the read endpoint (see Security).
- Root account credentials used for deployment (see Security).
- GET /tasks scans the whole table rather than using a scalable query

pattern - fine now, a real constraint later.

- The Lambda runtime, `nodejs20.x`, is on AWS's deprecation timeline (surfaced by `sam validate --lint` during CI/CD setup) and should be bumped to `nodejs22.x` in a future revision.
- No staging environment - every push to `main` deploys straight to the only environment that exists.
- The notification function only logs to CloudWatch rather than actually sending a real notification (email/Slack/SMS) - intentional scope limitation for this demo, but would be the next real feature to add.

Cost

This architecture's defining cost characteristic, compared directly against Project 1's EKS deployment, is that **nothing bills while idle**. Every component (Lambda, API Gateway, DynamoDB pay-per-request, SQS, S3, EventBridge) is billed per-request or per-resource-consumed rather than per-hour, and at this project's testing volume every component stayed within AWS's perpetual free tier. Project 1's EKS cluster, by contrast, billed roughly \$0.20-0.25/hour continuously regardless of whether the app was receiving any traffic, which is why it had to be manually deleted between work sessions to control cost. At low or bursty traffic - exactly this project's profile - the serverless architecture is straightforwardly cheaper and required no manual cost-management discipline to keep it that way.

Screenshots and Test Evidence

1. Successful task creation via the live API

```
PS ...\\serverless-tasks> Invoke-RestMethod -Uri $apiUrl/tasks `
    -Method POST -Headers $headers -Body $body

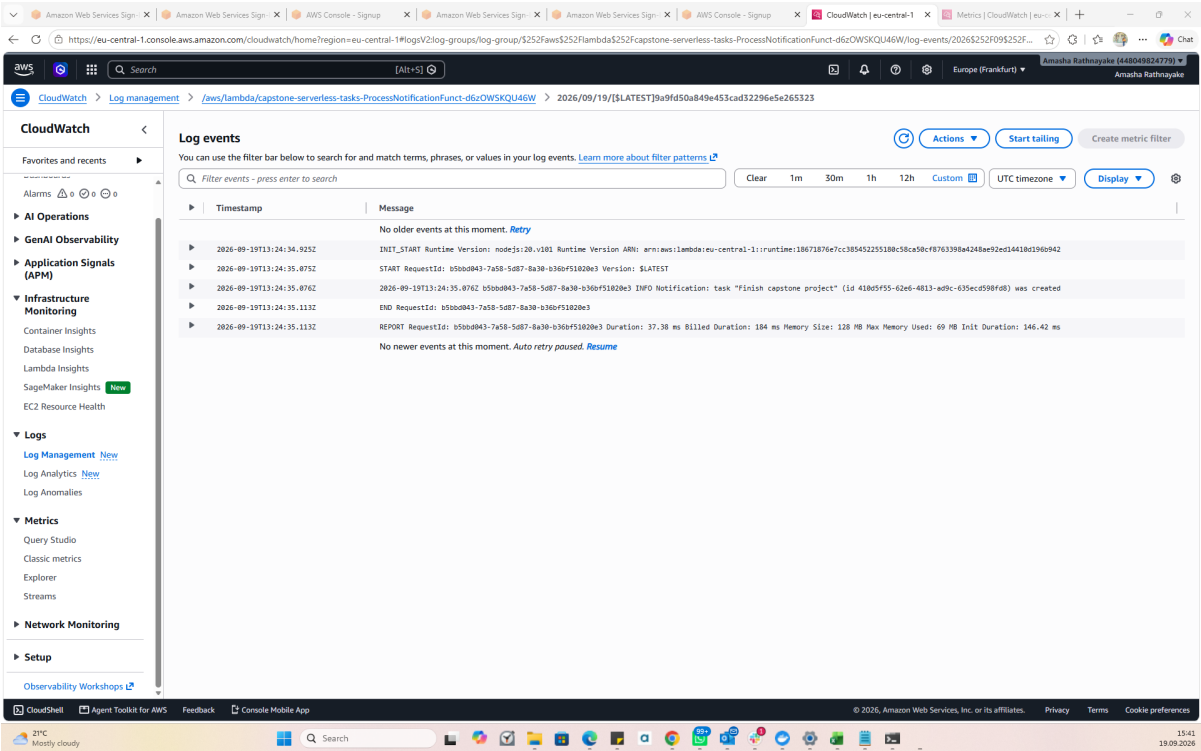
id                                     title                                done  createdAt
--                                     -
410d5f55-62e6-4813-ad9c-635ecd598fd8  Finish capstone project             False 2026-09-19T13:24:33.676Z

# $apiUrl = https://fe5m4bovta.execute-api.eu-central-1.amazonaws.com/prod
```

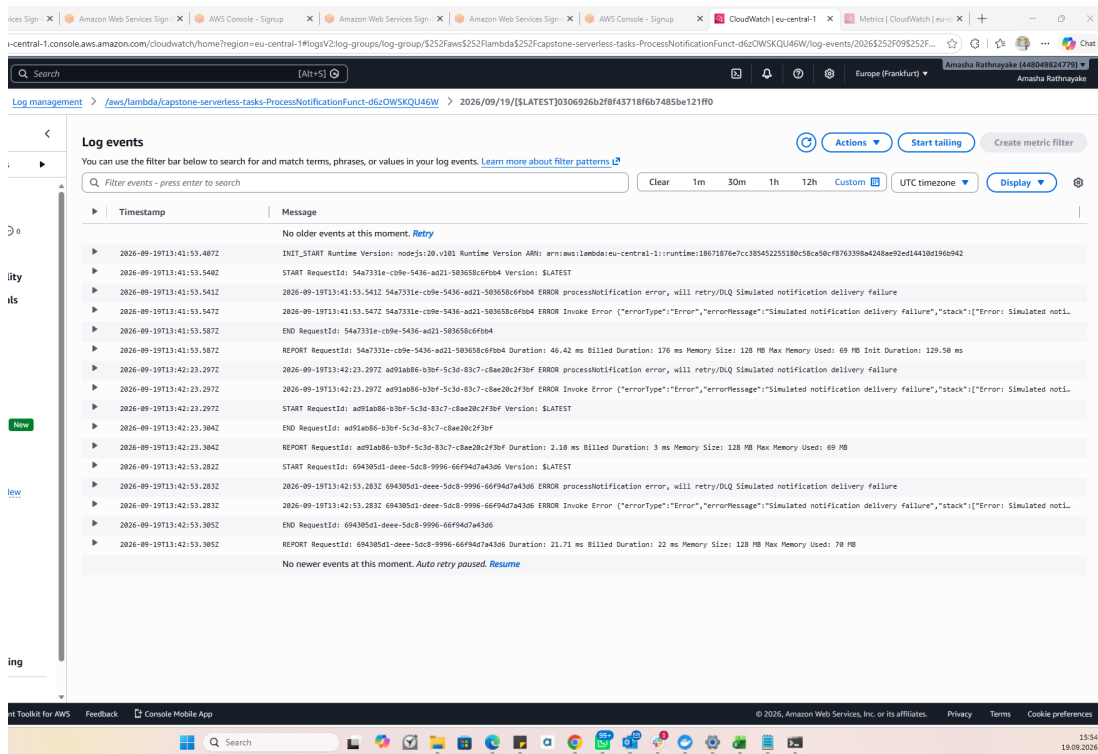
2. Listing tasks via the live API (confirms DynamoDB read path)

```
PS ...\\serverless-tasks> Invoke-RestMethod -Uri $apiUrl/tasks

createdAt                               id                                     done  title
-----
2026-09-19T13:24:33.676Z 410d5f55-62e6-4813-ad9c-635ecd598fd8 False Finish capstone project
```



3. SQS-triggered notification: CloudWatch Logs for processNotification, showing it fired automatically after task creation ('Notification: task "Finish capstone project" ... was created').



4. Retry behavior on failure: CloudWatch Logs showing 3 separate retry attempts (~30s apart) for a deliberately-failing task, each hitting 'Simulated notification delivery failure'.

5. Failed event confirmed in the Dead Letter Queue

```
PS ...\\serverless-tasks> aws sqs get-queue-attributes `
    --queue-url $dlqUrl --attribute-names ApproximateNumberOfMessages
{
  "Attributes": {
    "ApproximateNumberOfMessages": "1"
  }
}
# $dlqUrl = .../448049824779/task-notifications-dlq
# Confirms the message that failed 3x was correctly routed to the
# DLQ instead of being silently dropped.
```

6. Scheduled report function - manually invoked, CSV written to S3

```
PS ...\\serverless-tasks> aws lambda invoke `
    --function-name capstone-serverless-tasks-ExportReportFunction-zk8NwMJrBTPy out.json
{
  "StatusCode": 200,
  "ExecutedVersion": "$LATEST"
}

PS ...\\serverless-tasks> type out.json
{"reportKey":"reports/tasks-report-2026-09-19.csv","taskCount":2}

PS ...\\serverless-tasks> aws s3 ls s3://capstone-task-reports-448049824779/reports/
2026-09-19 15:55:47          217 tasks-report-2026-09-19.csv
```

All workflows

Showing runs from all workflows

Filter workflow runs

3 workflow runs		Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
✓	relax SAM validate lint level				main	4 minutes ago 1m 25s
Build, Test and Deploy Serverless App #3: Commit d472138 pushed by Amasha456						
✗	flatten project structure to repo root				main	8 minutes ago 39s
Build, Test and Deploy Serverless App #2: Commit 0a84ed4 pushed by Amasha456						
✗	fix workflow path - move .github to repo root				main	12 minutes ago 34s
Build, Test and Deploy Serverless App #1: Commit 3bb0e2b pushed by Amasha456						

7. CI/CD pipeline: GitHub Actions 'Build, Test and Deploy Serverless App' workflow, with the final run succeeding (green checkmark, 1m25s) - runs the unit test suite, validates the SAM template, builds, and deploys automatically on push.